

3. 包装类&泛型

【本节目标】

1. 以能阅读 java 集合源码为目标学习泛型
2. 了解包装类
3. 了解泛型

1. 包装类

在Java中，由于基本类型不是继承自Object，为了在泛型代码中可以支持基本类型，Java给每个基本类型都对应了一个包装类型。

1.1 基本数据类型和对应的包装类

基本数据类型	包装类
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

除了 Integer 和 Character，其余基本类型的包装类都是首字母大写。

1.2 装箱和拆箱

```
1 int i = 10;
2
3 // 装箱操作，新建一个 Integer 类型对象，将 i 的值放入对象的某个属性中
4 Integer ii = Integer.valueOf(i);
5 Integer ij = new Integer(i);
6
7 // 拆箱操作，将 Integer 对象中的值取出，放到一个基本数据类型中
8 int j = ii.intValue();
```

1.3 自动装箱和自动拆箱

可以看到在使用过程中，装箱和拆箱带来不少的代码量，所以为了减少开发者的负担，java 提供了自动机制。

```
1 int i = 10;
2
3 Integer ii = i;           // 自动装箱
4 Integer ij = (Integer)i;  // 自动装箱
5
6 int j = ii;               // 自动拆箱
7 int k = (int)ii;          // 自动拆箱
```

我们可以通过javap -c 查看字节码文件内容，观察装箱和拆箱的操作

【面试题】

下列代码输出什么，为什么？

```
1 public static void main(String[] args) {
2     Integer a = 127;
3     Integer b = 127;
4
5     Integer c = 128;
6     Integer d = 128;
7
8     System.out.println(a == b);
9     System.out.println(c == d);
10 }
```

2. 什么是泛型

一般的类和方法，只能使用具体的类型: 要么是基本类型，要么是自定义的类。如果要编写可以应用于多种类型的代码，这种刻板的限制对代码的束缚就会很大。----- 来源《Java编程思想》对泛型的介绍。

泛型是在JDK1.5引入的新的语法，通俗讲，泛型：**就是适用于许许多多类型**。从代码上讲，就是对类型实现了参数化。

3. 引出泛型

实现一个类，类中包含一个数组成员，使得数组中可以存放任何类型的数据，也可以根据成员方法返回数组中某个下标的值？

思路：

1. 我们以前学过的数组，只能存放指定类型的元素，例如：`int[] array = new int[10]; String[] strs = new String[10];`
2. 所有类的父类，默认为Object类。数组是否可以创建为Object?

代码示例：

```
1 class MyArray {
2     public Object[] array = new Object[10];
3
4     public Object getPos(int pos) {
5         return this.array[pos];
6     }
7     public void setVal(int pos, Object val) {
8         this.array[pos] = val;
9     }
10 }
11
12 public class TestDemo {
13     public static void main(String[] args) {
14         MyArray myArray = new MyArray();
15         myArray.setVal(0, 10);
16         myArray.setVal(1, "hello"); // 字符串也可以存放
17         String ret = myArray.getPos(1); // 编译报错
18         System.out.println(ret);
19     }
20 }
```

问题：以上代码实现后 发现

1. 任何类型数据都可以存放
2. 1号下标本身就是字符串，但是确编译报错。必须进行强制类型转换

虽然在这种情况下，当前数组任何数据都可以存放，但是，更多情况下，我们还是希望他只能够持有一种数据类型。而不是同时持有这么多类型。**所以，泛型的主要目的：就是指定当前的容器，要持有什么类型的对象。让编译器去做检查。**此时，就需要把类型，作为参数传递。需要什么类型，就传入什么类型。

3.1 语法

基础写法：

```
1 class 泛型类名称<类型形参列表> {
2     // 这里可以使用类型参数
```

```

3 }
4
5 class ClassName<T1, T2, ..., Tn> {
6 }

```

其他写法:

```

1 class 泛型类名称<类型形参列表> extends 继承类/* 这里可以使用类型参数 */ {
2     // 这里可以使用类型参数
3 }
4
5 class ClassName<T1, T2, ..., Tn> extends ParentClass<T1> {
6     // 可以只使用部分类型参数
7 }

```

上述代码进行改写如下:

```

1 class MyArray<T> {
2     public Object[] array = new Object[10];
3
4     public T getPos(int pos) {
5         return (T)this.array[pos];
6     }
7     public void setVal(int pos,T val) {
8         this.array[pos] = val;
9     }
10 }
11 public class TestDemo {
12     public static void main(String[] args) {
13         MyArray<Integer> myArray = new MyArray<>();//1
14         myArray.setVal(0,10);
15         myArray.setVal(1,12);
16         int ret = myArray.getPos(1);//2
17         System.out.println(ret);
18         myArray.setVal(2,"bit");//3
19     }
20 }

```

代码解释:

1. 类名后的 `<T>` 代表占位符, 表示当前类是一个泛型类
2. 了解: 【规范】类型形参一般使用一个大写字母表示, 常用的名称有:

- E 表示 Element
- K 表示 Key
- V 表示 Value
- N 表示 Number
- T 表示 Type
- S, U, V 等等 - 第二、第三、第四个类型

3. 注释1处，类型后加入 `<Integer>` 指定当前类型

4. 注释2处，不需要进行强制类型转换

5. 注释3处，代码编译报错，此时因为在注释2处指定类当前的类型，此时在注释4处，编译器会在存放元素的时候帮助我们进行类型检查。

4. 泛型类的使用

4.1 语法

```
1 泛型类<类型实参> 变量名;           // 定义一个泛型类引用
2 new 泛型类<类型实参>(构造方法实参); // 实例化一个泛型类对象
```

4.2 示例

```
1 MyArray<Integer> list = new MyArray<Integer>();
```

注意：泛型只能接受类，所有的基本数据类型必须使用包装类！

4.3 类型推导(Type Inference)

当编译器可以根据上下文推导出类型实参时，可以省略类型实参的填写

```
1 MyArray<Integer> list = new MyArray<>(); // 可以推导出实例化需要的类型实参为
Integer
```

5. 裸类型(Raw Type) (了解)

5.1 说明

裸类型是一个泛型类但没有带着类型实参，例如 `MyArrayList` 就是一个裸类型

```
1 MyArray list = new MyArray();
```

注意： 我们不要自己去使用裸类型，裸类型是为了兼容老版本的 API 保留的机制

下面的类型擦除部分，我们也会讲到编译器是如何使用裸类型的。

小结：

1. 泛型是将数据类型参数化，进行传递
2. 使用 `<T>` 表示当前类是一个泛型类。
3. 泛型目前为止的优点：数据类型参数化，编译时自动进行类型检查和转换

6. 泛型如何编译的

6.1 擦除机制

那么，泛型到底是怎么编译的？

1. 基本概念：

- 在编译时，Java 编译器会将泛型类型信息从代码中移除，这个过程就叫做类型擦除。
- 擦除后，泛型类型会被替换为其边界类型（通常是 Object）或者指定的类型。

2. 擦除过程：

- 将泛型参数替换为其边界或 Object。
- 在必要的地方插入类型转换以保持类型安全。
- 生成桥接方法以保持多态性。

3. 示例

上述代码在类型擦除后为如下代码：

擦除前：

```
1 class MyArray<T> {
2     public Object[] array = new Object[10];
3
4     public T getPos(int pos) {
5         return (T)this.array[pos];
6     }
7     public void setVal(int pos,T val) {
8         this.array[pos] = val;
9     }
10 }
```

擦除后：

```
1 class MyArray {
2     public Object[] array = new Object[10];
3
4     public Object getPos(int pos) {
5         return this.array[pos];
6     }
7
8     public void setVal(int pos, Object val) {
9         this.array[pos] = val;
10    }
11 }
```

6.2 关于桥接方法

- 泛型类型擦除可能导致子类方法和父类方法的签名不一致。
- 为了维护 Java 的多态性，需要桥接方法来确保子类方法能够正确覆盖父类方法。

```
1 public class Node<T> {
2     T data;
3
4     public void setData(T data) {
5         this.data = data;
6     }
7 }
8
9 public class StringNode extends Node<String> {
10    @Override
11    public void setData(String data) {
12        super.setData(data);
13    }
14 }
```

- 类型擦除后代码变为

```
1 public class Node {
2     Object data;
3
4     public void setData(Object data) {
5         this.data = data;
6     }
7 }
```

```

7 }
8
9 public class StringNode extends Node {
10     public void setData(String data) {
11         super.setData(data);
12     }
13 }

```

此时 `StringNode` 的 `setData` 方法并没有真正覆盖父类的 `setData` 方法（参数类型不同）。为了解决这个问题，编译器会在 `StringNode` 中生成一个桥接方法：

```

1 // 编译器生成的桥接方法
2 public void setData(Object data) {
3     setData((String) data);
4 }

```

7. 泛型的上界

在定义泛型类时，有时需要对传入的类型变量做一定的约束，可以通过类型边界来约束。

7.1 语法

```

1 class 泛型类名称<类型形参 extends 类型边界> {
2     ...
3 }

```

7.2 示例

```

1 public class MyArray<E extends Number> {
2     ...
3 }

```

只接受 `Number` 的子类型作为 `E` 的类型实参

```

1 MyArray<Integer> l1;           // 正常，因为 Integer 是 Number 的子类型
2 MyArray<String> l2;           // 编译错误，因为 String 不是 Number 的子类型

```

了解： 没有指定类型边界 `E`，可以视为 `E extends Object`

7.3 复杂示例

```
1 public class MyArray<E extends Comparable<E>> {
2     ...
3 }
```

- E必须是实现了Comparable接口的

8. 泛型方法

8.1 定义语法

```
1 方法限定符 <类型形参列表> 返回值类型 方法名称(形参列表) { ... }
```

8.2 示例

```
1 public class Util {
2     //静态的泛型方法 需要在static后用<>声明泛型类型参数
3     public static <E> void swap(E[] array, int i, int j) {
4         E t = array[i];
5         array[i] = array[j];
6         array[j] = t;
7     }
8 }
```

8.3 使用示例-可以类型推导

```
1 Integer[] a = { ... };
2 swap(a, 0, 9);
3
4 String[] b = { ... };
5 swap(b, 0, 9);
```

8.4 使用示例-不使用类型推导

```
1 Integer[] a = { ... };
2 Util.<Integer>swap(a, 0, 9);
3
```

```
4 String[] b = { ... };
5 Util.<String>swap(b, 0, 9);
```

9. 通配符

? 用于在泛型的使用，即为通配符

9.1 通配符解决什么问题

示例：

```
1 package www.bit.java.test;
2
3 class Message<T> {
4     private T message ;
5
6     public T getMessage() {
7         return message;
8     }
9
10    public void setMessage(T message) {
11        this.message = message;
12    }
13
14 }
15
16 public class TestDemo {
17     public static void main(String[] args) {
18         Message<String> message = new Message<>() ;
19         message.setMessage("比特就业课欢迎您");
20         fun(message);
21     }
22     public static void fun(Message<String> temp){
23         System.out.println(temp.getMessage());
24     }
25 }
```

以上程序会带来新的问题，如果现在泛型的类型设置的不是String，而是Integer。

```
1 public class TestDemo {
2     public static void main(String[] args) {
3         Message<Integer> message = new Message() ;
4         message.setMessage(99);
5         fun(message); // 出现错误，只能接收String
```

```

6    }
7    public static void fun(Message<String> temp){
8        System.out.println(temp.getMessage());
9    }
10 }

```

我们需要的解决方案：可以接收所有的泛型类型，但是又不能够让用户随意修改。这种情况就需要使用通配符"?"来处理

范例：使用通配符

```

1  public class TestDemo {
2      public static void main(String[] args) {
3          Message<Integer> message = new Message() ;
4          message.setMessage(55);
5          fun(message);
6      }
7      // 此时使用通配符"?"描述的是它可以接收任意类型
8      public static void fun(Message<?> temp){
9          System.out.println(temp.getMessage());
10     }
11 }

```

在"?"的基础上又产生了两个子通配符：

? extends 类：设置通配符上限

? super 类：设置通配符下限

9.2 通配符上界

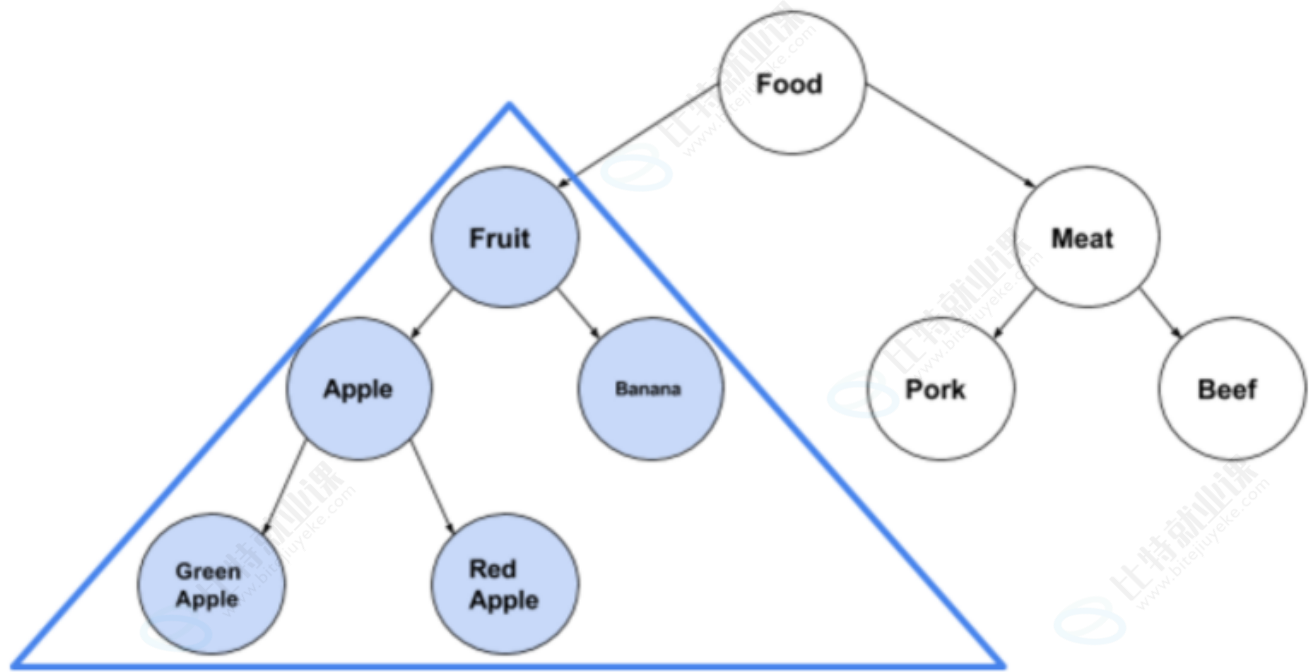
语法：

```

1  <? extends 上界>
2  <? extends Number> //可以传入的实参类型是Number或者Number的子类

```

Plate<? extends Fruit>



```
1 class Food {
2
3 }
4 class Fruit extends Food {
5
6 }
7 class Apple extends Fruit {
8
9 }
10 class Banana extends Fruit {
11
12 }
13 class Plate<T> { // 设置泛型
14     private T data ;
15
16     public T getData() {
17         return data;
18     }
19
20     public void setData(T data) {
21         this.data = data;
22     }
23 }
24 }
25
26 public class TestDemo {
27     public static void main(String[] args) {
```

```

28     Plate<Apple> plate = new Plate<>() ;
29     plate.setData(new Apple());
30     fun(plate);
31
32     Plate<Banana> plate2 = new Plate<>() ;
33     plate2.setData(new Banana());
34     fun(plate2);
35
36 }
37 // 此时使用通配符 "?" 描述的是它可以接收任意类型，但是需要是 Fruit 或者 Fruit 的子类
38 public static void fun(Plate<? extends Fruit> temp){
39     //temp.setData(new Banana()); //error
40     //temp.setData(new Apple()); //error
41     System.out.println(temp.getData());
42 }
43 }

```

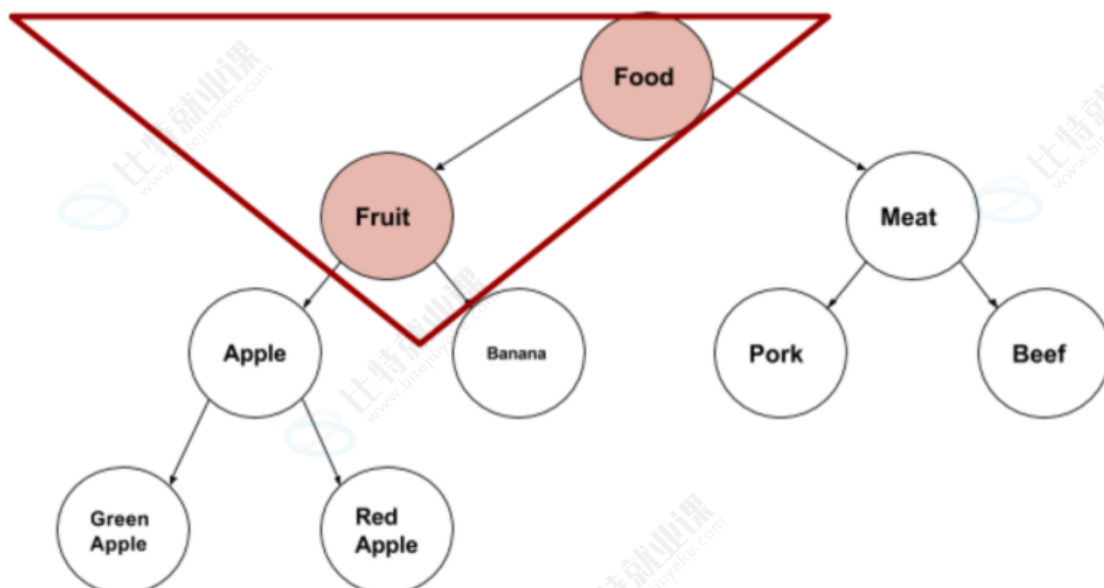
此时无法在fun函数中对temp进行添加元素，因为temp接收的是Fruit和他的子类，此时存储的元素应该是哪个子类无法确定。所以添加会报错！

9.3 通配符下界

语法

- 1 <? **super** 下界>
- 2 <? **super** Integer> //代表 可以传入的实参的类型是Integer或者Integer的父类类型

Plate<? super Fruit>



```
1 class Food {
2
3 }
4 class Fruit extends Food {
5
6 }
7 class Apple extends Fruit {
8
9 }
10 class Plate<T> {
11     private T plate ;
12
13     public T getData() {
14         return plate;
15     }
16
17     public void setData(T plate) {
18         this.plate = plate;
19     }
20 }
21 public class TestDemo {
22     public static void main(String[] args) {
23         Plate<Fruit> plate1 = new Plate<>();
24         plate1.setData(new Fruit());
25         fun(plate1);
26
27         Plate<Food> plate2 = new Plate<>();
28         plate2.setData(new Food());
29         fun(plate2);
30     }
31
32     public static void fun(Plate<? super Fruit> temp){
33         // 此时可以修改!! 添加的是Fruit 或者Fruit的子类
34         temp.setData(new Apple()); //这个是Fruit的子类
35         temp.setData(new Fruit()); //这个是Fruit的本身
36         //Fruit fruit = temp.getPlate(); 不能接收, 这里无法确定是哪个父类
37         System.out.println(temp.getData()); //只能直接输出
38     }
39 }
```

